DATABASE SYSTEMS ENGINEERING & DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)

Hospital Management & Appointment System - MediDesk

Implemented as "MediDesk" — Hospital Operations Platform

2520090016 **P. Rahul**

2520080005 **A. Lakshmi Sai Teja**

Problem Statement

- Manual paperwork and long patient waiting times
- Appointment booking and tracking handled inconsistently
- Data entry errors and difficulty accessing patient records
- Poor coordination between billing, pharmacy, and lab
- No centralized way to monitor hospital-wide activity
- Result: hospitals need one connected system, not several disconnected ones

Objectives

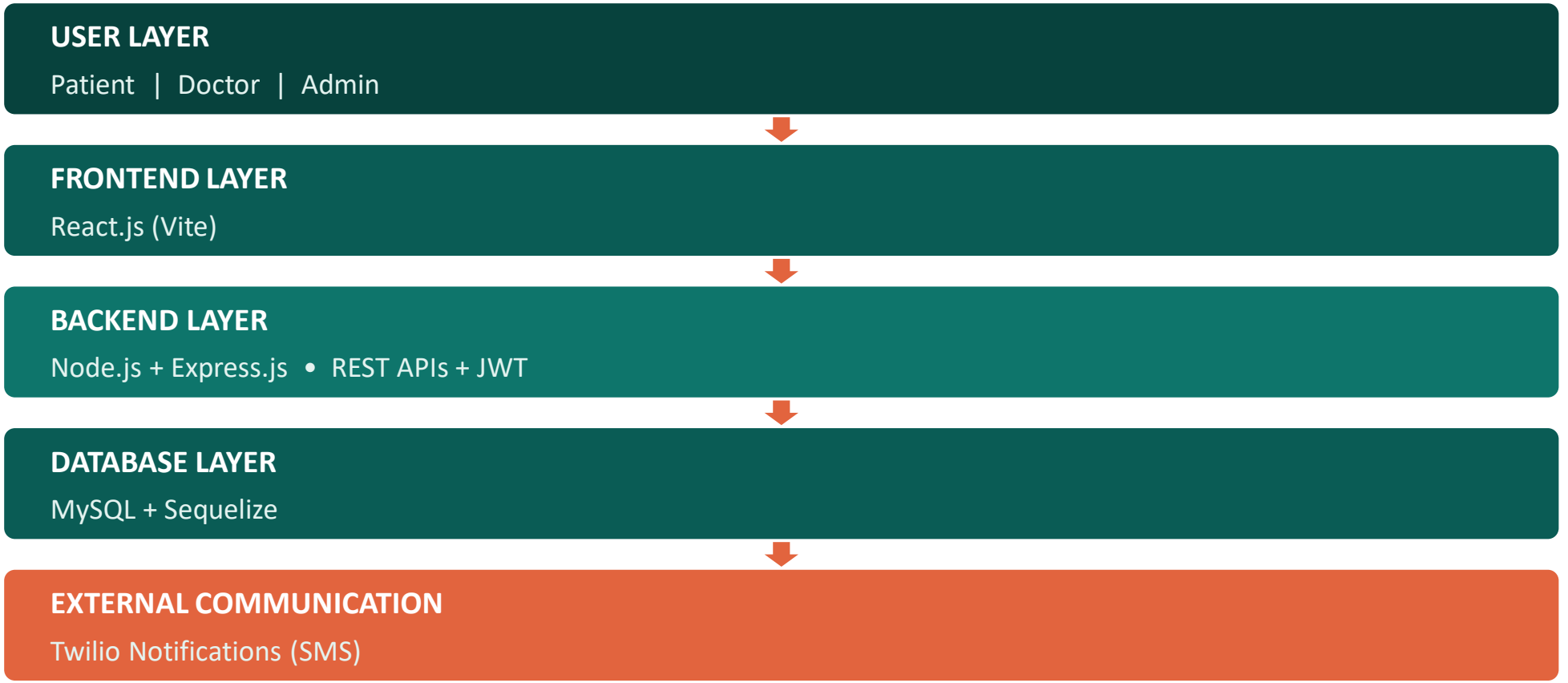
- Centralize hospital operations on a single web-based platform
- Simplify patient registration and appointment booking
- Give doctors fast access to Electronic Medical Records (EMR)
- Reduce waiting time through token & queue management
- Streamline billing, pharmacy, and laboratory coordination
- Reduce paperwork and minimise manual data errors

Proposed System

- One centralized web-based platform for all hospital operations
- Patients register, book appointments, and track status online
- Doctors access EMR and manage their schedules
- Token & queue management reduces waiting time
- Billing, pharmacy, and lab data stay linked to patient records
- Admin dashboard gives staff a centralized view of activity

System Architecture

Layered Architecture Diagram



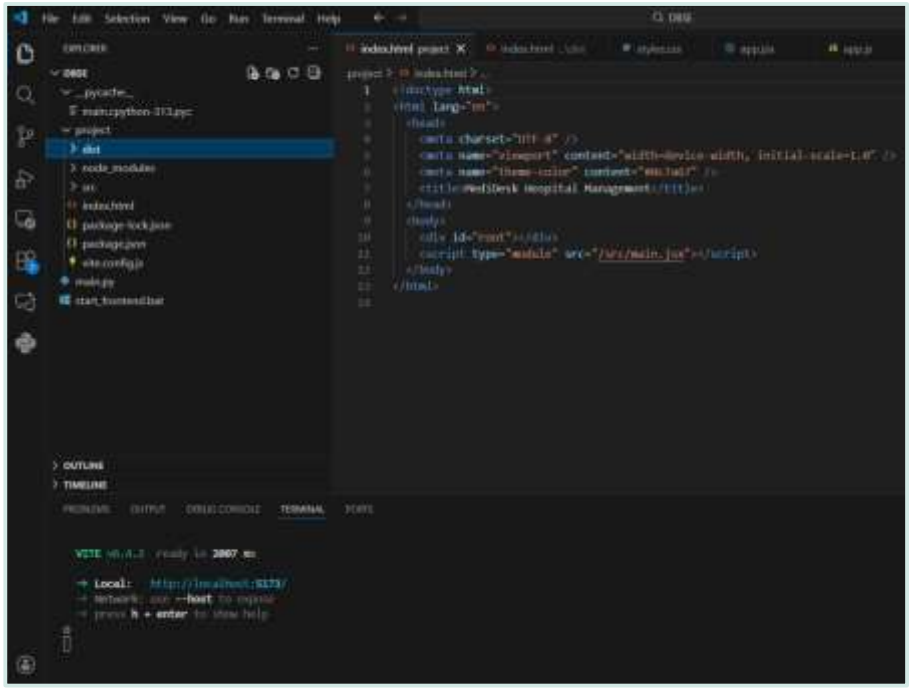
Modules & Functionalities

- Patient Registration
- Appointment Booking
- Doctor Schedule Mgmt
- Electronic Medical Records
- Token & Queue Management
- Billing & Insurance
- Pharmacy & Inventory
- Laboratory / Test Reports
- Admin Dashboard
- Notifications (SMS)

Frontend Design

- Three role-based portals: Admin, Doctor, Patient
- Each portal opens to a personalized Dashboard
- Sidebar navigation for Appointments, EMR, Billing, Pharmacy, Lab Reports
- Forms for appointment booking, billing & consultation notes
- Built with React.js using Vite for fast development

Development Environment

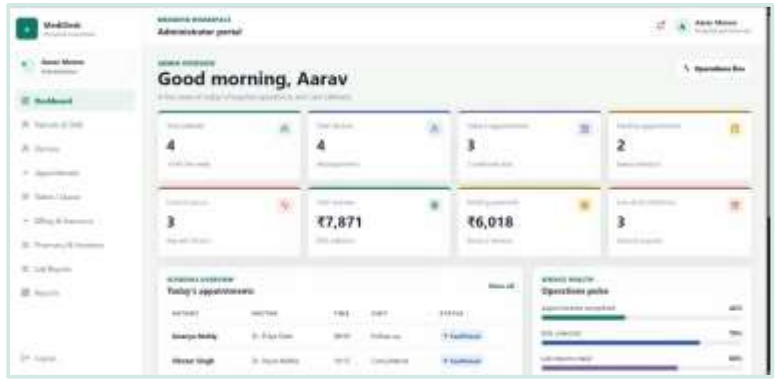


React + Vite dev server running MediDesk locally

Frontend Implementation — Screenshots

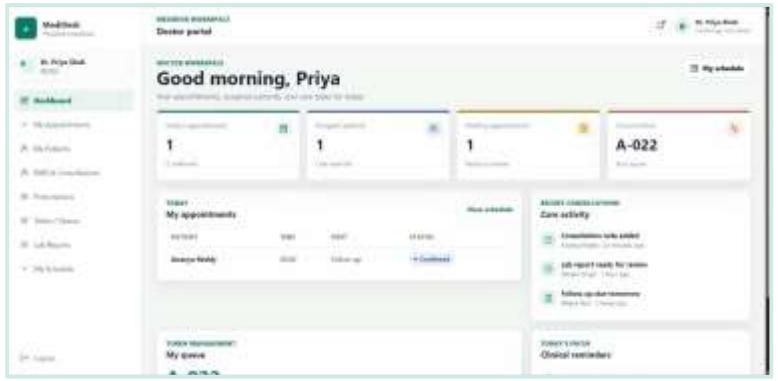
Live dashboards for each user role, built and running in MediDesk

Admin Portal



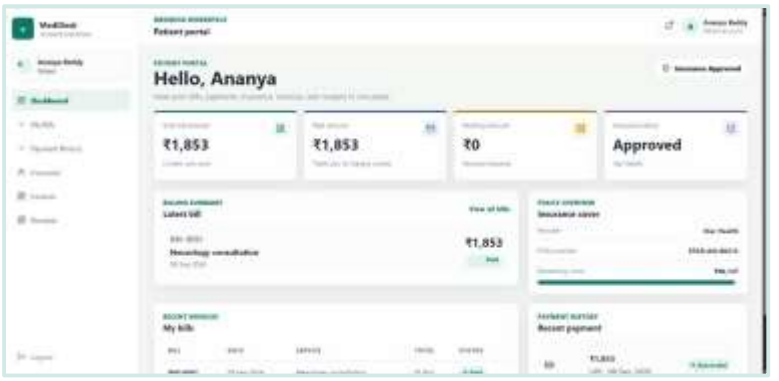
Live stats, schedule & queue overview

Doctor Portal



Today's appointments & patient consultations

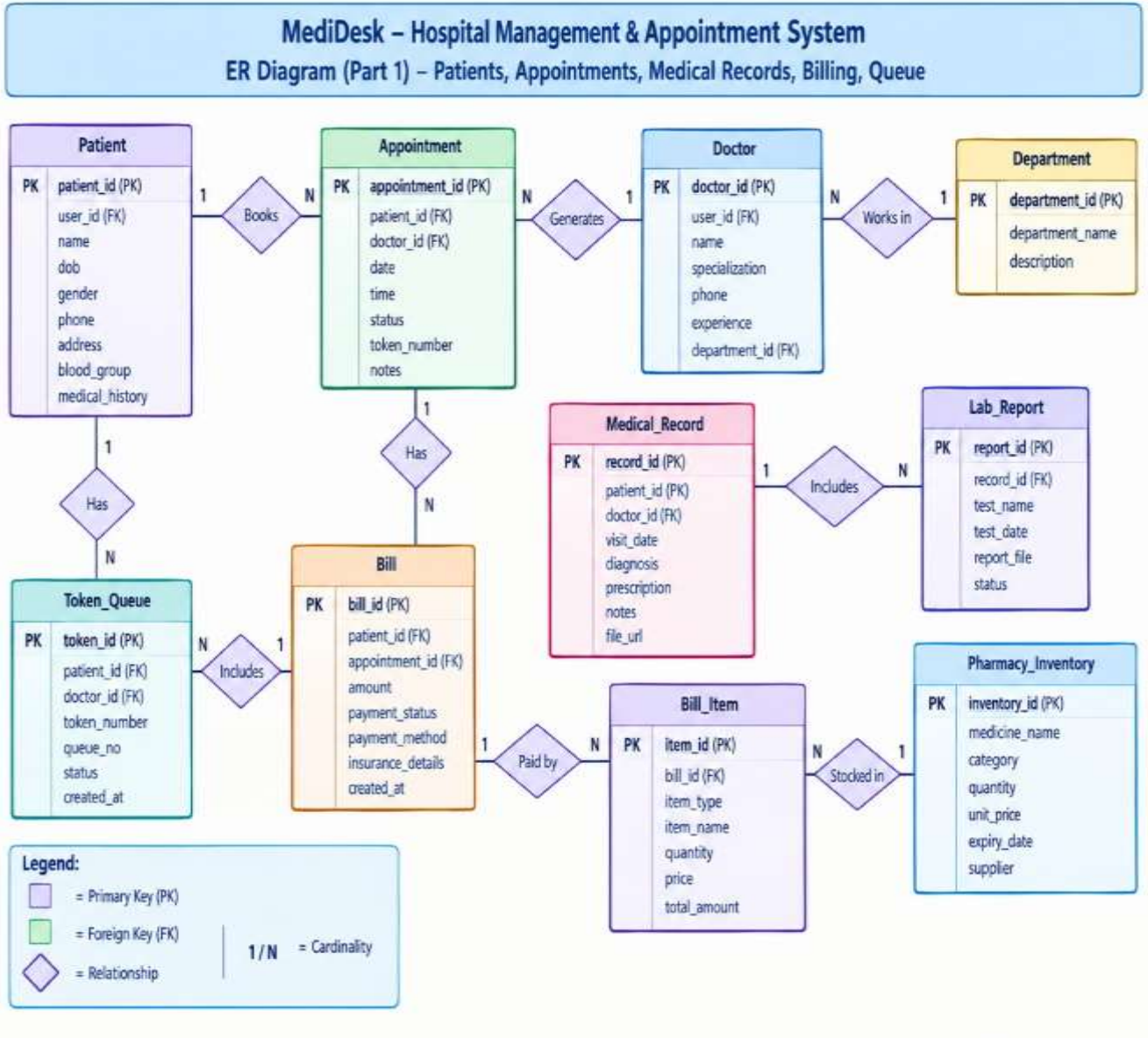
Patient Portal



Bills, payments & insurance overview

All three portals share consistent MediDesk branding, navigation, and card-based UI styling.

Database Design — ER Diagram



Key Points

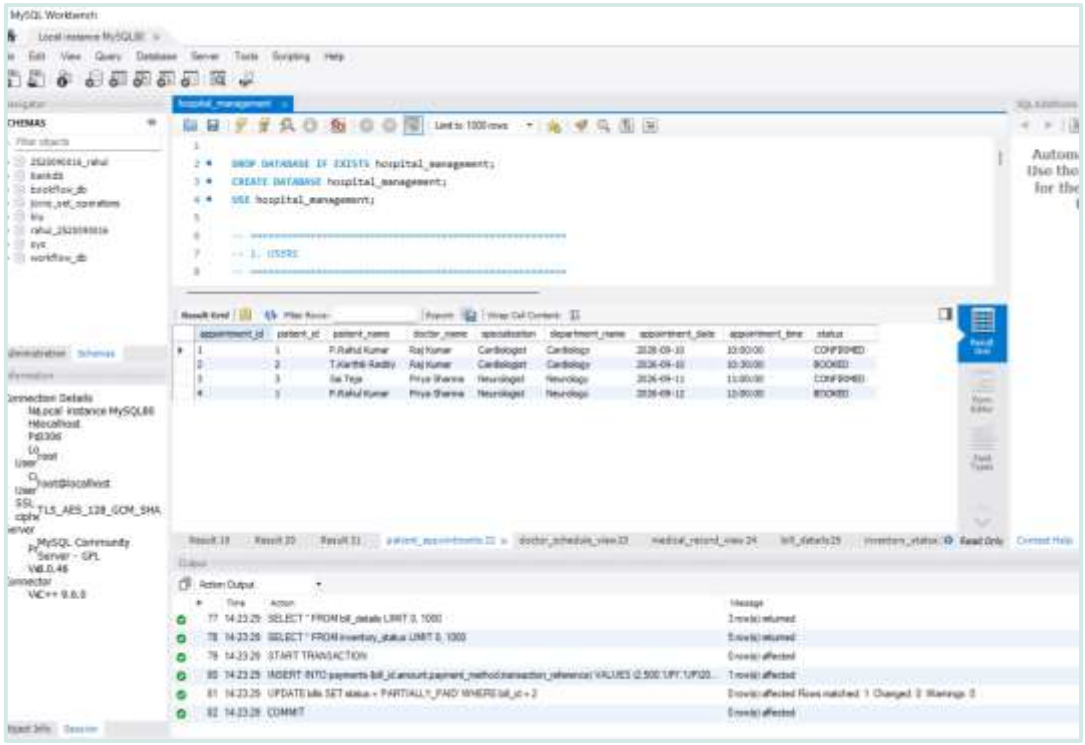
- MySQL relational database: hospital_management
- ER modelling with normalised tables
- Primary & foreign key constraints link entities
- Joins, transactions, views & stored logic used

Database Schema & Sample Records

Views Implemented

- patient_appointments — appointment records
- doctor_schedule_view — doctor availability
- medical_record_view — patient EMR data
- bill_details — billing & invoices
- inventory_status — pharmacy stock levels
- Transactions verified: INSERT → UPDATE → COMMIT

MySQL Workbench — Sample Data



Literature Survey & Research Gap

Existing Approach	Major Features	Gap Identified	MediDesk Contribution
Traditional / Manual HMS	Registration, paper records, manual appointments	Time-consuming, errors	Centralized digital workflow
Appointment Systems	Online appointment booking	Limited hospital-wide integration	Appointment + Token/Queue management
Electronic Hospital Systems	EHR and patient records	Other departments may be separate	EMR + Billing + Pharmacy + Lab integration
Existing HMS Platforms	Multiple hospital modules	Complex or broad enterprise systems	Simple role-based Patient/Doctor/Admin dashboards
MediDesk	Integrated hospital operations	—	One platform connecting appointments, EMR, queue, billing, pharmacy, laboratory and administration

Current Progress

- Frontend built with React.js (Vite) — Admin, Doctor & Patient portals live
- MySQL database designed and implemented (hospital_management schema)
- Core views built for appointments, billing, inventory & medical records
- Sample data inserted and tested with transactional updates
- Backend REST API integration (Node.js/Express, JWT) in progress
- UI styled and functional across all three role-based dashboards

Challenges / Issues Faced

- Structuring a relational schema that keeps billing, payments & insurance consistent
- Keeping token/queue status synced across portals
- Implementing consistent role-based access across three dashboards
- Designing a UI simple enough for patients, detailed enough for admins

Future Work

- Complete backend REST API integration with JWT authentication
- Add SMS notifications via Twilio
- Build an analytics dashboard for hospital-wide insights
- Explore a telemedicine / video consultation module
- Deploy using Docker for production readiness

Conclusion

- Centralizes hospital operations on a single, connected platform
- Frontend (3 role-based portals) and database implementation show real progress
- Remaining work: backend API integration, notifications, and deployment

Thank You